

Observability: OpenTelemetry for Agentic AI

Module reference -- ADAR Knowledge Academy -- Agonomia Labs Product Tour

Agentic AI systems make multiple decisions and call multiple services on the way to a single answer. Without tracing, a slow or wrong response is a black box. ADAR instruments every service with OpenTelemetry so that a single request can be replayed, step by step, after the fact.

Anatomy of a trace

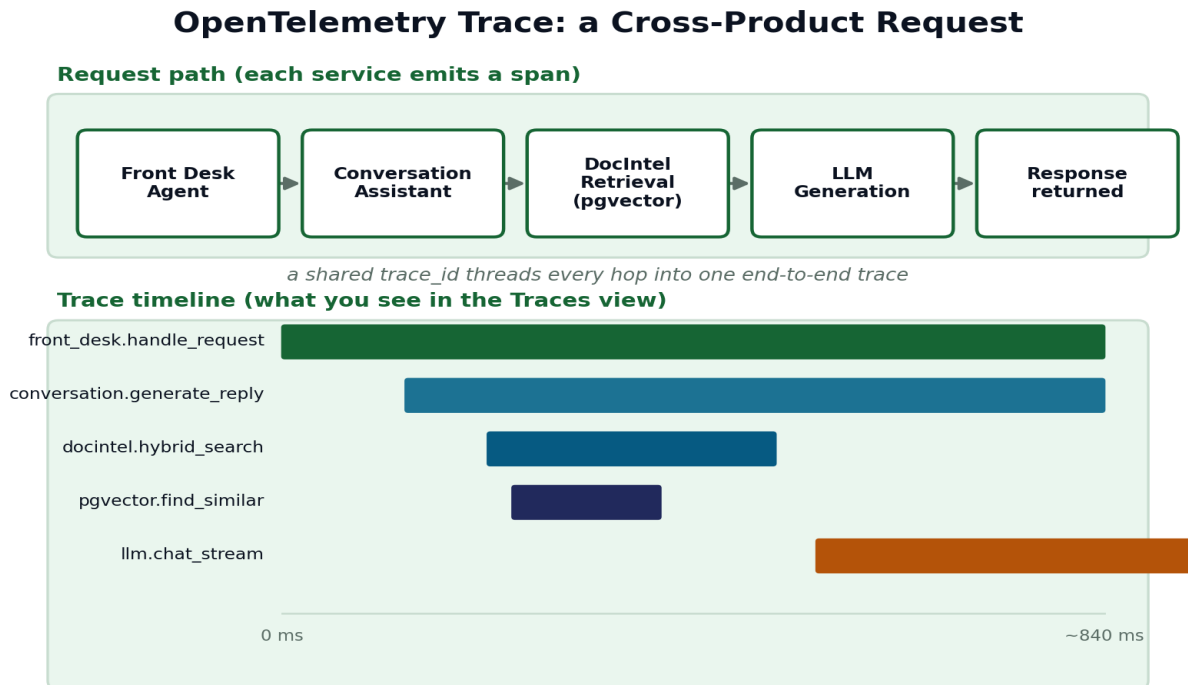


Figure 1. A cross-product request -- Front Desk to Conversation Assistant to DocIntel retrieval to LLM generation -- and the resulting trace timeline.

Spans and traces

- A **span** is one unit of work with a name, a start time, a duration, and structured attributes (for example: which document IDs were searched, how many chunks were retrieved).
- A **trace** is the tree of spans produced by one end-to-end request, linked by a shared `trace_id` that propagates across service boundaries -- even across separate Cloud Run services.
- Child spans nest inside their parent's time window, so the trace view makes it immediately visible which step of a slow request was actually the bottleneck.

What this looks like in practice

In the example above, a Front Desk booking conversation calls the Conversation Assistant, which calls DocIntel's hybrid search (itself built on a pgvector similarity query), and finally streams a response from the LLM. Every one of those hops is its own span inside one trace -- so a support engineer (or an AI Tutor answering "why was this slow") can see exactly where the ~840 ms went, without any custom logging having been written by hand.

Why this matters for trustworthy AI

Tracing is what turns "the AI said something" into "the AI retrieved these specific sources, in this order, and generated this response from them, in 840 ms." That same instrumentation is what lets this Knowledge Academy's own AI Tutor cite its sources back to you -- the retrieval step you see traced here is the same step that produces the citations under every Tutor answer.